

Design and Development of a Technology-Agnostic NFR Testing Framework

Introducing the framework and discussing the future of load testing in Agile software development

E. Whiting

University of Houston – Clear Lake, College of Science
and Engineering
erik@erikwhiting.com

S. Datta

University of Houston – Clear Lake, College of Science
and Engineering
datta@uhcl.edu

ABSTRACT

Testing the non-functional requirements (NFR) of a system is particularly complicated and time-consuming. Challenges in this area are compounded when the system is developed under some offspring of Agile methodologies, which favor iterative development and rapid feedback from extensive testing. The authors of this paper build upon previous work investigating the common challenges and solutions cited in recent peer-reviewed research on this topic to design and build a tool consolidating many of the concepts found in this investigation. The tool is known as LuluPerfTest (LPT) and is an NFR testing framework meant to plug into continuous integration (CI) systems to run NFR tests configured with a JSON script. This allows developers and testers to build maintainable and minimally complex automated NFR test scripts. This study explains the challenges inherent in NFR testing in Agile software development and presents how LPT confronts those challenges. It aims to explain LPT and invite collaboration among other testing, verification, and validation researchers to create an open sources software (OSS) solution to the problems of NFR testing in Agile software development projects.

CCS CONCEPTS

• **Software and its engineering**; • **Software creation and management**; • **Software verification and validation**;

KEYWORDS

Software testing, performance testing, non-functional requirements

ACM Reference Format:

E. Whiting and S. Datta. 2022. Design and Development of a Technology-Agnostic NFR Testing Framework: Introducing the framework and discussing the future of load testing in Agile software development. In *2022 The 5th International Conference on Software Engineering and Information Management (ICSIM) (ICSIM 2022)*, January 21–23, 2022, Yokohama, Japan. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3520084.3520092>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICSIM 2022, January 21–23, 2022, Yokohama, Japan

© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9551-9/22/01...\$15.00
<https://doi.org/10.1145/3520084.3520092>

1 INTRODUCTION

Agile software development (ASD) [1] refers to a collection of development methodologies which share 12 principles. Methodologies under the ASD share characteristics such as frequent delivery of working software (iterative development), consistent velocity, and open communication [2–4]. These characteristics are the result of common practices among the different methodologies, most relevant of which to this presentation is the dedication to effective software testing.

Testing is a broad discipline in software engineering [5, 11, 12] and its specific implementation differs among projects and teams. Software testing can include various testing activities including unit, integration, exploratory, smoke, or usability testing, and many more. This paper focuses on nonfunctional requirements (NFR) testing—a set of testing activities which aim to assert the system under test’s (SUT) reliability, scalability, elasticity, security, and more (these are often referred to colloquially as “the -ilities”). In ASD terms, this type of testing exists in quadrant 4 (Q4) of the Agile testing quadrants [6] as illustrated in figure 1

NFR testing is uniquely challenging in the context of ASD projects [13, 14], and as such, there is a wide variety of research in this area. After reviewing this research, the authors set out to develop an NFR testing framework to help testers and other software professionals create and maintain a suite of NFR tests in a way conducive to Agile methodologies. The authors analyzed several studies and concluded that in order to accomplish this goal, the framework would have to be a technology agnostic tool facilitating

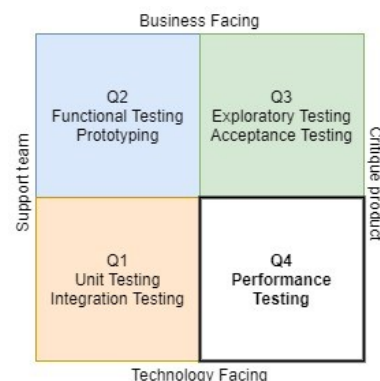


Figure 1: Model of Agile Testing Quadrants as introduced by Brian Marick and popularized by Lisa Crispin and Janet Gregory

rapid test script development and easily plugging into continuous integration (CI) tools [15].

The NFR testing framework developed became known as LuluPerfTest (LPT) and currently exists as an open-source software project [26]. LPT consists of a Performance Test model and a test runner that uses that model to conduct a performance test. The Performance Test model is comprised of a Configurations model, many Use Case models, and many Monitor models. The Use Case model provides LPT users a means to simulate the behavior of SUT users, while the Monitor model provides LPT users the means to designate and monitor system components while the Use Cases are running. The Configurations model simply allows LPT users to designate how the test run is logged (console output, hitting API endpoint, or populating a database). These models are populated with a JSON script that LPT consumes to coordinate running use cases and monitoring system components. The intended use is that the JSON script will be included in the SUT’s test suite and run in a CI system such as Circle CI, Jenkins, or other similar products.

The remaining sections of this paper will be structured as follows: section 2 expands upon some of the challenges in NFR testing in Agile, section 3 speaks to the design and implementation of the LPT NFR testing framework and how it aims to address some of the problems mentioned in section 2. Section 4 discusses limitations of the framework while section 5 discusses future work to address those limitations. Finally, section 6 concludes the paper.

2 CHALLENGES WITH NFR TESTING IN ASD

There is much documentation in the recent peer-reviewed research around the challenges associated with fitting non-functional requirements (NFR) testing into agile software development (ASD) projects, which are most accurately summed up in [16–18]. Most of the challenges cited in the literature can be classified into three broad categories: complexity, expense, and time.

2.1 Complexity, Expense, and Time

Complexity is due to the fact many of the metrics gathered by NFR tests require both specialized and broad knowledge to properly analyze. Furthermore, there are challenges around what is called the “test oracle problem” [19], which is the challenge of determining correct or desired output for some given input—a problem compounded in NFR testing [16].

NFR testing expense is due to several factors, including the expense of experienced performance engineers [17], hardware or cloud resources required to conduct NFR testing effectively, and the expense of existing tools [8]. Contributing to expense related challenges is the time-consuming nature of NFR testing. Such testing involves simulating users over an extended period of time and gathering relevant hardware metrics, then conducting complex analysis on those metrics. Test runs can often last many hours and often have to be run outside normal business times. Together, the time consuming and complex nature of NFR testing also adds to its cost [9].

2.2 Challenges Specific to Agile Software Development

The unique challenges of NFR testing are especially pronounced in ASD projects. ASD requires rapid and meaningful feedback from a test suite which is difficult given the time requirement for effective NFR testing [7, 10]. Another contributing factor to ASD-specific challenges in NFR testing is the fact that metrics gathered from performance tests are most meaningful when the environment in which they are gathered is similar to the SUT’s production environment [27]. That is to say that NFR testing often requires completely integrated and near-finished systems if test results are to provide meaningful insights. This is in contrast to ASD methodologies which favor the delivery of working software in successive iterations.

3 DESIGN, DEVELOPMENT, AND EVALUATION OF LPT

After analyzing the recent literature, the authors set out to address some of the challenges highlighted in section 2 by creating an open-source non-functional requirements testing framework called LuluPerfTest (LPT), which can be viewed or downloaded at the repository linked in [26]. Specifically, LPT aims to mitigate challenges related to time and complexity, which would have a positive effect on cost. This section describes the authors’ goals for LPT, and how it is designed and developed to achieve those goals.

3.1 Goals

Broadly, LPT aims to improve the field of software quality by alleviating challenges inherent in NFR testing; both in industry and academia. To accomplish this broad goal, the authors set two criteria the framework must meet: The software must be free and open source, and the framework must mitigate the time-consuming, complex, and expensive nature of NFR testing in Agile software development. The rest of section 3.1 elaborates on these goals. Section 3.3 will elaborate on how LPT meets these goals.

3.1.1 Free Open-Source Software. In order to provide the most value to the software testing community, the LPT framework must be free open-source software (FOSS). FOSS has been shown to facilitate better identification of requirements as well as provide better software quality than closed-source software [28]. Also, the authors wish to invite collaboration and communication in this area among all interested software quality professionals and researchers. FOSS is the best way to do this because the software system does not exist behind a pay wall and the system is accessible to all members of the public. Most importantly, the software uses the MIT license, allowing anyone to obtain the source code of the system, modify it, contribute to it, or study it.

3.1.2 Addressing NFR Testing Challenges. The authors’ main goal in developing LuluPerfTest to address NFR testing challenges in ASD is to reduce the time and complexity of the test development process, which also indirectly positively affects cost [9]. To accomplish this, LPT strives to provide a technology-agnostic tool to test script developers to customize load and system monitoring. Technologic agnosticism means that testers will not have to learn new languages or tools and can leverage existing test resources

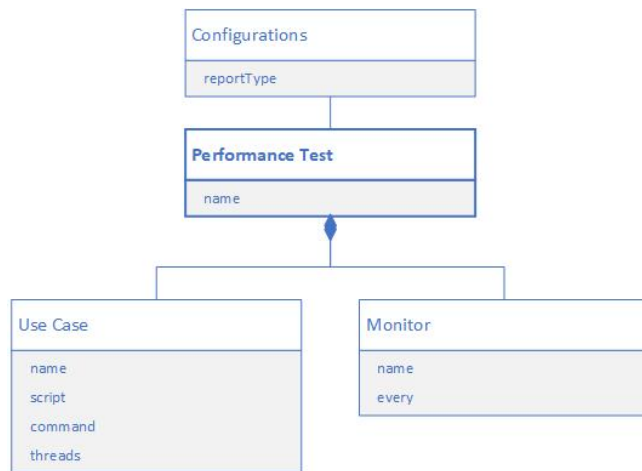


Figure 2: LPT Performance Test Model

when developing NFR tests, minimizing time requirements and complexity. Finally, to fit into ASD practices, LPT strives to be both version-controllable (e.g., can be managed with Git) and pluggable with CI systems, a common ASD tool [15].

3.2 Design and Development

3.2.1 Intended Usage. LuluPerfTest (LPT) is designed to be used by test script developers, which will be referred to as testers for the rest of this section. LPT assumes that testers will be testing against an already-built environment; that is, LPT does not handle resource provisioning or provide a means of running environment setup scripts. When developing an NFR test to run against a system, testers must first decide what kind of load—simulated users—to put the system under, and what system metrics to monitor such as CPU or memory usage. Furthermore, with LPT, testers have the option of adding behavior to the load with test automation scripts, allowing for further customization. Once the load, behavior, and metrics to monitor have been defined, the tester can then populate LPT’s Performance Test model, visualized in Figure 2

The Performance Test model consists first of a Configuration model. The Configuration model, at this time, is only used to designate how test case results are reported. As of now, test scripts can be reported by logging results to the system output; future work will include hitting an API or being inserted directly into a database. Next, the Performance Test has many Use Case models. Use Cases run against the SUT and can be as simple as a bash command, or as complicated as a multi-step automated user-interface (UI) test. The location of the script to be ran is recorded in the Use Case script attribute, and the command to run it is located in its command attribute. The threads attribute designates how many threads will run the script. Finally, the Performance Test model has many Monitor models. Monitors are system metrics to be reported while test scripts are running on the SUT. The name attribute serves to select which metric of the system to monitor, and testers can provide either “memory,” “CPU,” or “disk space” without further configuration to monitor those metrics on the SUT. The attribute every designates how many milliseconds to wait between each

gathering of the metric. Additionally, and not pictured in Figure 2, users can create a custom Monitor with any name and populate a command attribute that consists of a terminal command to extract the desired metric.

The Performance Test model is populated via a JSON script. The following segment shows an example:

```
// example_script.json
{
  "Performance Test": {
    "name": "default",
    "configurations": {
      "reportType": "string"
    },
    "useCases": [
      {
        "name": "Explore Albums",
        "script": "path/to/script.py",
        "command": "python",
        "threads": "2"
      },
      {
        "name": "Explore Bands",
        "script": "path/to/script.rb",
        "command": "ruby",
        "threads": "2"
      }
    ],
    "monitors": [
      {
        "name": "memory", "every": "500"
      },
      {
        "name": "CPU", "every": "500"
      },
      {
        "name": "log file size", "every": "500", "command": "du log.txt"
      }
    ]
  }
}
```

The example script above shows a Performance Test named “default” configured to print test results to the system output. The script then defines two use cases. The first is a use case named “Explore Albums” which is modeled in the Python script located at “path/to/script.py.” This datapoint, combined with the “command”: “python” datapoint, means that that this use case will be started by running “python path/to/script.py”. Finally, the threads attribute indicates that this use case will run on two threads, simulating two users performing the actions defined in the test script.

Notice also that the next Use Case model is similar to the first but is a ruby file. The command is “ruby” and LPT will start this script by running “ruby path/to/script.rb”. Such a scenario may be uncommon, but this example is to show that LPT is technology agnostic. Again, the “threads”: “2” line indicates that this script will run on two threads, simulating two users.

Next, the script defines three instances of the Monitor model. The first and second monitors are named memory and CPU. Using these names tells LPT to use its built-in means of checking the appropriate system metric. The values are gathered via Java’s `OperatingSystemMXBean`, so testers do not have to worry about operating-system-specific implementations of gathering these metrics. The third metric—the size of log.txt—is not provided out of the box

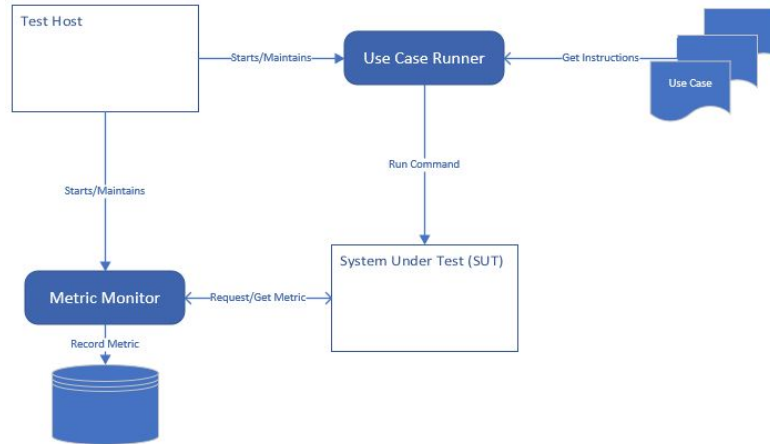


Figure 3: LPT High Level Architecture

via LPT. Notice that the script includes a command attribute here which is populated with a bash command for getting a file’s size. Each of these metrics is gathered every half second, as indicated by the “every”: “500” key-value pair.

One way to add this test script to the SUT’s test suite is by placing a JAR of LPT in the SUT’s test harness along with the JSON script above. Then, within the SUT’s CI configuration file, testers designate the time to run the performance test by making the JAR executable and then calling the JAR and passing in the JSON file. The following YAML example was tested by the authors in Travis CI:

```

- chmod 777 ./tests/LPT/LuluPerfTest.jar
- java -jar ./tests/LPT/LuluPerfTest.jar
  $home/tests/LPT/basic_test.json
  
```

This will make LPT consume the JSON file, populating the Performance Test model and running the test. This particular implementation will output test results in the build server. An example of the performance test output is shown below:

```

{threadId=22, data=43.113, threadName=Memory, monitorIteration=37}
{threadId=22, data=43.047, threadName=Memory, monitorIteration=38}
{threadId=22, data=43.125, threadName=Memory, monitorIteration=39}
{threadId=22, data=43.085, threadName=Memory, monitorIteration=40}
{threadId=22, data=43.096, threadName=Memory, monitorIteration=41}
  
```

The lines above were generated from running an LPT script monitoring local memory usage while running a few automated test scripts against a locally running web application. The data key is reporting the actual memory usage while the threadName indicates what metric is being monitored.

3.2.2 System Architecture. At a high level, LPT consists of two main components, the Metric Monitor and the Use Case Runner. Simultaneously, the Metric Monitor pings the SUT for its metrics at the intervals configured in the test script and records them for later use. The Use Case runner consumes automation scripts and runs them against the SUT. Figure 3 depicts a high-level architecture of LPT.

LPT is written in Java and makes heavy use of multithreading libraries in order to simulate many users while also monitoring system metrics. Both the UseCase and MetricMonitor packages



Figure 4: Use Case Diagram

implement Java’s Runnable interface. Figure 4 shows a model of the UseCase package.

Figure 5 shows a model of the MetricMonitor package.

Please note, the source code is available for download via GitHub and readers are encouraged to go to the link in [26] to view LPT’s source code implementation.

3.3 Evaluating the System

After development, LPT was evaluated by considering the authors’ two goals for the system: to be free and open-source software, and to provide a means for addressing NFR testing challenges in ASD. The system succeeds easily in its first goal by being publicly hosted on GitHub, so this report will not go into further detail about that goal. Regarding the goal of addressing NFR testing challenges in ASD, LPT accomplishes this in a few ways.

First, users of LPT can write complete performance tests in just a few lines of JSON. For example, the JSON script in section 3.2.1 defines a customized load and measures three different system

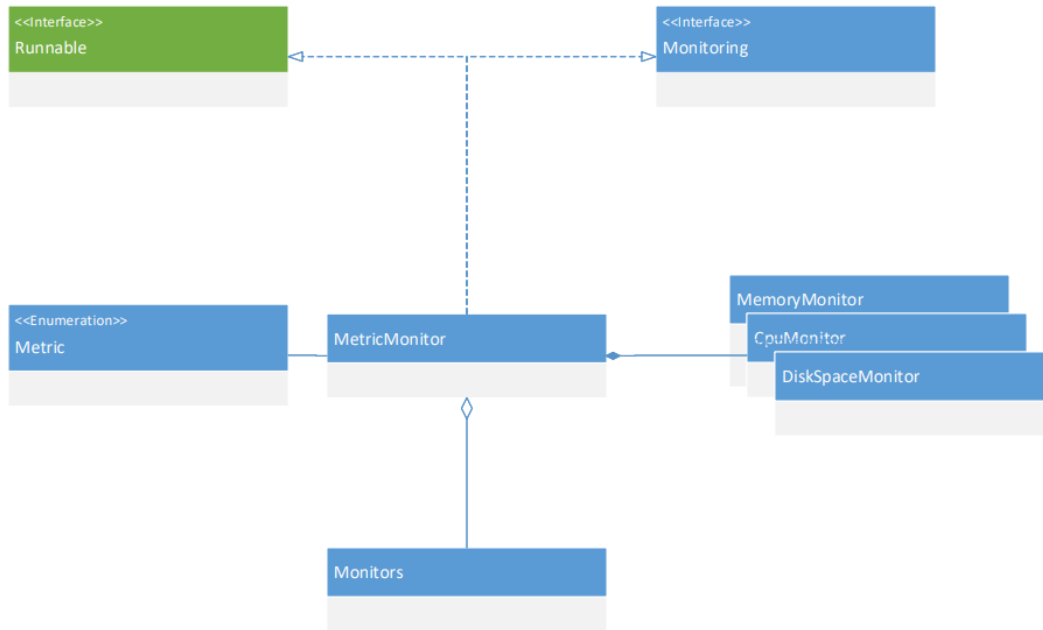


Figure 5: MetricMonitor module

metrics. This performance test required only 33 lines of JSON, 16 of which are only brackets and/or commas. Further customization of the load or monitors requires a trivial amount of work. The authors believe this facilitates rapid test script development and proves LPT addresses complexity and time-related challenges inherent in NFR testing in ASD.

Additionally, LPT’s load customization features do not require any specific programming language or tool, so testers may leverage their existing resources to create performance tests. This minimizes the time spent configuring the load behavior for performance tests. Furthermore, the metric monitoring features are also technology agnostic from the perspective of LPT. Testers can use the shell language of their SUT’s operating system to gather customized metrics. The technological agnosticism of LPT reduces complexity and time in NFR test script development because it allows testers to rely on tools which they are already competent.

Finally, as a tool, LPT can naturally fit into Agile environments and existing tools. First, because the test scripts are written in JSON, they can be version controlled and managed as part of a test suite. More importantly, LPT is pluggable with CI systems, a common tool in ASD projects [15]. The authors tested LPT by writing a test script for a mock web-application, maintaining it via version control, and configuring it to run in a Travis CI build server with the rest of the mock application’s tests. LPT performed as expected inside the CI server and required only a few lines of YAML in the Travis CI configuration file to do so.

4 LPT LIMITATIONS

Although the authors consider LuluPerfTest (LPT) successful in meeting its initial goals, they do not consider the framework done. There are a few limitations in LPT that must be overcome in order

for it to be a universally useful NFR testing framework. The main challenge for LPT is that it is poorly designed for coordinating distributed tests. Currently, one instance of an LPT performance test can run on one machine. Because it makes heavy use of threading, it is unlikely that one machine can generate a production-like load.

This challenge is compounded when considered in the context of continuous integration. Typically, the system hosting the SUT is also hosting the test suite. In a traditional CI configuration, LPT would be running on the same infrastructure as the application it’s testing. This is equivalent to, say, a server simultaneously hosting a web application as well as all of the web application’s users. This is rarely indicative of a real-world implementation, meaning the system metrics would not be realistic.

Additionally, as mentioned before, LPT only reports test results to the console, so the results can only be viewed in the CI’s output. Next steps in LPT development involve extending the framework to allow testers to send test results to an API endpoint or database. Such a feature would allow testers to present and analyze data from individual test runs as well as track results of many runs over time. Finally, LPT is limited by its pseudo-DSL. Currently, the Performance Model is populated via a JSON script which users must type. This means that users may mistype keywords, make mistakes with indentation, or other syntactic level mistakes that could break the test. Furthermore, IDEs are not able to catch these mistakes and LPT does not currently have any error reporting mechanisms. The authors’ plans for addressing these limitations is discussed in the next section

5 FUTURE WORK

The most pertinent task for the LPT framework is to field test the software in industry. As of writing this, it functions as a load-testing

framework that will effectively preform load tests as defined by users via the JSON DSL. However, without proper field testing, the authors can only speculate on if it fulfils its goals, particularly if it fits well into Agile projects. As such, future work will involve soliciting agile practitioners and software quality assurance personnel to implement the framework into their daily work and report back to the authors their evaluations and thoughts. From there, a design-science methodology will be used to further develop the LPT framework to a robust and useful framework.

One of the main motivations for creating the LPT framework is to promote conversations around non-functional requirements testing in Agile software development. LPT is a free and open-source software tool, and its maintainer (the first author) welcomes all contributors and users. The authors hope that creating this FOSS tool and sharing research will facilitate efforts to further the field of NFR testing.

On a more practical side, the framework itself can improve in many places, as noted in section 4. To address issues regarding distributed testing, the authors hope to bring in other researchers and developers with fresh perspectives. Recently, it has been proposed that a model for coordinating multiple instances of LPT in a cloud environment is the most promising approach for addressing this problem.

Regarding the JSON scripts, the first author intends to develop a proper domain-specific language (DSL) for writing performance tests with LPT. Such a DSL would allow for syntactic enforcement and would mean IDEs could be configured to provide suggestions and syntax highlighting. An alternative to this would be to provide a UI that generates a JSON file like the example in section 3.2.1.

6 CONCLUSION

This paper introduces LuluPerfTest, a free and open-source non-functional requirements testing framework. The tool is designed to address challenges related to NFR testing in Agile software development such as complexity, time, and cost. The framework addresses these challenges providing a simple Performance Test domain model that can be populated with a few lines of JSON. The model is then consumed by the LPT test runner to run highly customizable performance tests. The framework aims not only to alleviate many of the challenges related to NFR testing in ASD, but to invite and facilitate collaboration and conversation among researchers and professionals in addressing NFR testing challenges. The authors welcome all inquiries and collaborators.

REFERENCES

- [1] Manifesto for Agile Software Development, <https://agilemanifestor.org>, last accessed 2021/5/5.
- [2] Guang-yong, H.: Study and Practice of import Scrum agile software Development. In: 2011 IEEE 3rd International Conference on Communication Software and Networks, pp. 217-220. IEEE, Xi'an.
- [3] Newkirk, J., Martin, R.: Extreme Programming in Practice. In: Addendum to the 2000 proceedings of the conference on Object-oriented programming, systems, languages, and applications (Addendum), pp. 25-26. ACM, 2000.
- [4] Shafiq, M., Waheed, U.: Documentation in Agile Development A Comparative Analysis. In: 2018 IEEE 21st International Multi-Topic Conference (INMIC), pp. 1-8. IEEE, 2018.
- [5] DeMillo, R. A.: Software Testing, Encyclopedia of Computer Science. Pp. 1645-1649 (2003).
- [6] Crispin, L., Gregory, J.: Agile Testing: A Practical Guide for Testers and Agile Teams. Addison-Wesley Professional, Boston (2009).
- [7] Ferme, V., Pautasso, C.: A Declarative Approach for Performance Tests Execution in Continuous Software Development Environments. In: 2018 ACM/SPEC International Conference on Performance Engineering, pp. 261-272. ACM, Germany (2018).
- [8] Omar Portillo-Dominguez, A., Wang, M., Murphy, J., Magoni, D., Mitchell, N., Sweeney, P., Altman, A.: towards an automated approach to use expert systems in the performance testing of distributed systems. In: Proceedings of the 2014 Workshop on Joining AcadeMiA and Industry Contributions to Test Automation and Model-Based Testing, pp. 22-27. ACM, Jamaica (2014).
- [9] Schulz, H., Okanovic, D., van Hoorn, A., Ferme, V., Pautasso, C.: Behavior-driven Load Testing Using Contextual Knowledge – Approach and Experiences. In: Proceedings of the 2019 ACM/SPEC International Conference on Performance Engineering, pp. 265-272. ACM, India (2019).
- [10] Field, T., Chatley, R., Wei, D.: Software Performance Testing in Virtual Time. In: ICPE '18: Companion of the 2018 ACM/SPEC International Conference on Performance Engineering, pp. 173-174. ACM, Germany (2018)
- [11] Hellman, T., Chokshi, A., Abad, Z., Pratte, S., Maurer, F.: Agile Testing: A Systematic Mapping across Three Conferences: Understanding Agile Testing in the XP/Agile Universe, Agile, and XP Conferences. In: 2013 Agile Conference, pp. 32-41. IEEE, US (2013)
- [12] Hellman, T., Sharma, A., Ferreira, J., Maurer, F.: Agile Testing: Past, Present, and Future – Charting a Systematic Map of Testing in Agile Software Development. In 2012 Agile Conference, pp. 55-63. IEEE, USA (2012)
- [13] Paetsch, F., Eberlein, A., Maurer, F.: Requirements Engineering and Agile Software Development. In Proceedings of the 12th IEEE International Workshop on Enabling Technologies: Infrastructure of Collaborative Enterprises, pp. 308-313. IEEE, Austria (2003)
- [14] Camacho, C., Marczak, S., Cruzes, D.: Agile Team Members Perceptions on Non-functional Testing: Influencing Factors from an Empirical Study. In 2016 11th International Conference on Availability, Reliability and Security (ARES), pp. 582-589. IEEE, Austria (2016)
- [15] Stolberg, S.: Enabling Agile Testing Through Continuous Integration. In 2009 Agile Conference, pp. 369-374. IEEE, USA (2009)
- [16] Segura, S., Troya, J., Duran, A., Ruiz-Cortes, A.: Performance metamorphic testing: motivation and challenges. In Proceedings of the 39th International Conference on Software Engineering: New Ideas and Emerging Results Track, pp. 7-10. ACM, Argentina (2017)
- [17] Walter, J., Van Hoorn, A., Koziol, H., Okanovic, D., Kounev, S.: Asking "What?", Automating the "How?": The Vision of Declarative Performance Engineering. In Proceedings of the 7th ACM/SPEC on International Conference on Performance Engineering, pp. 91-94. ACM, Netherlands (2016)
- [18] Bezemer, C., Eismann, S., Ferme, V., Grohmann, J., Heinrich, R., Jamshidi, P., Shang, W., van Hoorn, A., Villavicencio, M., Walter, J., Willnecker, F.: How is Performance Addressed in DevOps? In Proceedings of the 2019 ACM/SPEC International Conference on Performance Engineering, pp. 45-50. ACM, India (2019)
- [19] Jahangirova, G.: Oracle problem in software testing. In Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 444-447. ACM, USA (2017)
- [20] Bertolino, A., De Angelis, G., Gallego, M., Garcia, B., Gortazar, F., Lonetti, F., Marchetti, E.: A Systematic Review on Cloud Testing. In ACM Computing Surveys 93:1-42. (2019)
- [21] Chawla, N., Kumar, D., Sharma, D.: Enabling Performance Management During Cloud Applications Migration. In 4th International Conference on Information Systems and Computer Networks (ISCON), pp. 196-201. IEEE, India (2019)
- [22] Dlugi, M., Brunnert, A., Krcmar, H.: Model-Based Performance Evaluations in Continuous Delivery Pipelines. In Proceedings of the 1st International Workshop on Quality-Aware DevOps, pp. 25-26. ACM, Italy (2015)
- [23] Xu, H., Su, C., Wu, S., Tang, D.: Survey of Testing Methods of O2O Catering Platform. In Proceedings of the 2019 3rd High Performance Computing and Cluster Technologies Conference, pp. 220-224. ACM, China (2019)
- [24] Raigoza, J., Thakkar, R.: Browser Performance of JavaScript Framework. In 2016 International Conference on Computational Science and Computational Intelligence (CSCI), pp. 1420-1421. IEEE, USA (2016)
- [25] Silva, A., Araujo, T., Nunes, J., Perkusich, M., D Lorenzo, E., Almeida, H., Perkusich, A.: A Systematic Review on the Use of Definition of Done on Agile Software Development Projects. In Proceedings of the 21st International Conference on Evaluation and Assessment in Software Engineering, pp. 364-373. ACM, Sweden (2017)
- [26] LuluPerfTest Repository. <https://github.com/erik-whiting/LuluPerfTest>
- [27] Kattenpur, A., Manbiar, M.: Service Demand Modeling and Prediction with Single-User Performance Tests. In Proceedings of the 9th Annual ACM India Conference, pp. 31-41. ACM, India (2016)
- [28] Khanjani, A., Sulaiman, R.: The aspects of choosing open source versus closed source. In IEEE Symposium on Computers & Informatics, pp. 646-649. IEEE, Malaysia (2011)